```
import os
import sys
import random
import glob
import shutil
import numpy as
import tqdm
import time
import matplotlib as mpl
import matplotlib.pyplot as plt
import matplotlib.patches as patches

from skimage import utils
from skimage import io
from skimage.transform import display_images
from skimage.transform import rgb2gray
from skimage.model import bag

import torch

TempDataDir = os.environ
os.environ['TORR_STORAGE_DIRECTORY'] = './.D'

# Dataset directory
DATASET_DIR = 'Dataset/proposed_data'

# Inference configuration
from mrcnn.inference.inferenceConfig import
config.display()
Configurations:
BACKBONE                    resnet50
BACKBONE_STRIDES            [4, 8, 16, 32, 64]
BATCH_SIZE                  1
BBOX_STD_DEV                [0.1 0.1 0.2 0.2]
COMPUTE_BACKBONE_SHAPE      None
DETECTION_MAX_INSTANCES     400
DETECTION_MIN_CONFIDENCE    0
DETECTION_NMS_THRESHOLD     0.3
FPN_CLASSIF_FC_LAYERS_SIZE  1024
GPU_COUNT                   1
GRADIENT_CLIP_NORM          5.0
IMAGE_CHANNEL_COUNT         3
IMAGE_MAX_DIM               512
IMAGE_MIN_DIM               512
IMAGE_MIN_SCALE             0
IMAGE_RESIZE_MODE           square
IMAGES_PER_GPU              1
LEARNING_MOMENTUM           0.9
LEARNING_RATE               0.001
LOSS_WEIGHTS                {'rpn_class_loss': 1.0, 'rpn_bbox_loss': 1.0, 'mrcnn_class_loss': 1.0, 'mrcnn_bbox_loss': 1.0, 'mrcnn_mask_loss': 1.0}
MAX_GT_INSTANCES            100
MEAN_PIXEL                  [0. 0. 0.]
MINI_MASK_SHAPE             (56, 56)
NAME                        shapes
NUM_CLASSES                 [10 13 20 24 30 34 41 52]
PRE_NMS_LIMIT               6000
POOL_SIZE                   7
POST_NMS_ROIS_INFERENCE     2000
POST_NMS_ROIS_TRAINING      2000
PRE_NMS_LIMIT               6000
RPN_ANCHOR_RATIOS           [0.5, 1, 2]
RPN_ANCHOR_SCALES           (8, 16, 32, 64, 128)
RPN_ANCHOR_STRIDE           1
RPN_BBOX_STD_DEV            [0.1 0.1 0.2 0.2]
RPN_NMS_THRESHOLD           0.7
RPN_TRAIN_ANCHORS_PER_IMAGE 256
STEPS_PER_EPOCH             1000
TOP_DOWN_PYRAMID_SIZE       256
TRAIN_BN                    False
TRAIN_ROIS_PER_IMAGE        128
USE_MINI_MASK               True
USE_RPN_ROIS                True
VALIDATION_STEPS            50
WEIGHT_DECAY                0.0001

# Device to load the neural network on
# Useful if you might have more than 1 gpu
# machine, in which case you CPU and leave the
# GPU free for other jobs
DEVICE = "/cpu:0"  # /cpu:0 or /gpu:0

# Inspect the model in training or inference modes
# values: 'inference' or 'training'
# TODO: code for 'training' test mode not ready yet
TEST_MODE = "inference"

def get_ax(rows=1, cols=1, size=16):
    """Return a Matplotlib Axes array to be used in
    all visualizations in the notebook. Provide a
    central point to control graph sizes.
    Adjust the size attribute to control how big to render images
    To use one subplot
    fig, ax = plt.subplots(rows, cols, figsize=(size*cols, size*rows))
    return ax
    return ax
```

```
# Load validation dataset
dataset = zum.LoadDataset()
dataset.load_medical(DATASET_DIR, "val")
dataset.prepare()
print("Images: {}\nClasses: {}".format(len(dataset.image_ids), dataset.class_names))
```

```
Found 55 images in dataset/proposed_data/val/images
Adding image ID: 0
Adding image ID: 1
Adding image ID: 10
Adding image ID: 11
Adding image ID: 12
Adding image ID: 13
Adding image ID: 14
Adding image ID: 15
Adding image ID: 16
Adding image ID: 17
Adding image ID: 18
Adding image ID: 19
Adding image ID: 2
Adding image ID: 20
Adding image ID: 21
Adding image ID: 22
Adding image ID: 23
Adding image ID: 24
Adding image ID: 25
Adding image ID: 26
Adding image ID: 27
Adding image ID: 28
Adding image ID: 29
Adding image ID: 3
Adding image ID: 30
Adding image ID: 31
Adding image ID: 32
Adding image ID: 33
Adding image ID: 34
Adding image ID: 35
Adding image ID: 36
Adding image ID: 37
Adding image ID: 38
Adding image ID: 39
Adding image ID: 4
Adding image ID: 40
Adding image ID: 41
Adding image ID: 42
Adding image ID: 43
Adding image ID: 44
Adding image ID: 45
Adding image ID: 46
Adding image ID: 47
Adding image ID: 48
Adding image ID: 49
Adding image ID: 5
Adding image ID: 50
Adding image ID: 51
Adding image ID: 52
Adding image ID: 53
Adding image ID: 54
Adding image ID: 6
Adding image ID: 7
Adding image ID: 8
Adding image ID: 9
Images: 55
Classes: ['BG', 'Cone']
```

```
# Create model in inference mode
DEVICE = "/cpu"
with tf.device(DEVICE):
    model = modelZoo.MaskRCNN(mode="inference",
                              model_dir=LOG_DIR,
                              config=config)

# Path to a specific weights file
weights_path = "logs/medical20181227T1417mask_rcnn_cone_medical_0001.h5"

# If last the last models you trained
weights_path = model.find_last()

# Load weights
print("Loading weights ", weights_path)
model.load_weights(weights_path, by_name=True)
```

```
Loading weights  logs/medical20181227T1417mask_rcnn_cone_medical_0001.h5
No starting from scratch !
```

```
# image_id = random.choice(dataset.image_ids)
# image, image_meta, gt_class_id, gt_bbox, gt_mask =
#     modelZoo.load_image_gt(dataset, config, image_id, use_mini_mask=False)
# log("original_image", image)
# print("Image ID: {}.{} ({}) {}".format(info['source'], info['id'], image_id,
# log("image_meta", image_meta)
print("Original image shape: ", modelZoo.parse_image_meta(image_meta)["original_image_shape"][0])

# Run object detection
results = model.detect_medical([np.expand_dims(image, 0), np.expand_dims(image_meta, 0)], verbose=1)

# Display results
ax = get_ax(1)
r = results[0]
log("gt_class_id", gt_class_id)
log("gt_bbox", gt_bbox)
log("gt_mask", gt_mask)

# Compute AP over range 0.5:0.95 and print it
AP, precisions, recalls, overlaps = utils.compute_ap_range(
                                    gt_bbox, gt_class_id, gt_mask,
                                    r['rois'], r['class_ids'], r['scores'], r['masks'],
                                    verbose=1)
#print(gt_mask.shape,r['masks'].shape)
print("mAP: {:.3f}".format(AP))
from skimage.measure import find_contours
visibility = np.ones(r['rois'].shape[0], dtype=int)
# visibility = np.where(r['class_ids'] > 0, 1, 0)
N = r['rois'].shape[0]
if not np.shape(r['rois'])[0]:
    print("No instances to display!")
else:
    assert r['rois'].shape[0] == r['masks'].shape[-1] == r['class_ids'].shape[0]
    for i in range(N):
        if visibility[i] == 1:
            color = tuple(random_colors(1))
            ax.add_patch(patches.Rectangle((r['rois'][i][1], r['rois'][i][0]),
                         r['rois'][i][3] - r['rois'][i][1],
                         r['rois'][i][2] - r['rois'][i][0],
                         fill=False,
                         edgecolor=color,
                         linewidth=2,
                         alpha=0.7))
    if not r['class_ids'][i] > 0:
        continue
    mask = r['masks'][:, :, i]
    if not np.shape(mask)[0]:
        continue
    else:
        verts = np.fliplr(find_contours(mask, 0.5)[0])
        p = patches.Polygon(verts, facecolor="none", edgecolor=color)
        ax.add_patch(p)
ax.imshow(image[:,:,:])
plt.savefig("./image_with_mask.jpg")
plt.show()
```

```
def compute_batch_ap(dataset, image_ids, verbose=None):
    APs = []
    for image_id in image_ids:
    image = dataset.load_image(image_id)
    mask, cls = dataset.load_mask(image_id)
    from progressbar import *
    # add this if the top if not processed
    bar = ProgressBar()
    for image_id in range(image_ids):
        # Load image
        image, image_meta, gt_class_id, gt_bbox, gt_mask =
            modelZoo.load_image_gt(dataset, config,
                                   image_id, use_mini_mask=False)
    # Run object detection
    results = model.detect_medical([np.ex-ndims(), image_meta[np.newaxis], verbose=0)
    # Compute AP over range 0.5:0.95
    # Nx6 -> Nx1
    ap = utils.compute_ap(gt_bbox, gt_class_id, gt_mask,
                          r['rois'], r['class_ids'], r['scores'], r['masks'],
                          verbose=0)
    APs.append(ap)
    if verbose:
        info = dataset.image_info[image_id]
        print("{:3} {}   AP: {:5.3f}".format(
            image_id, np.mean(APs), ap))
    return APs, MeanAcc, IoMatrixAP0, MeanMIOU
    # return APs
from skimage.measure import distance
from skimage.measure import mean_squared_error
from skimage.metrics import ssim, psnr
# Compute VOC AP over all images
# gt_bbox_list = []
# gt_class_id_list = []
# gt_mask_list = []
# for image_id in dataset.image_ids:
# image, image_meta, gt_class_id, gt_bbox, gt_mask =
# modelZoo.load_image_gt(dataset, config,
# image_id, use_mini_mask=False)
# gt_bbox_list.append(gt_bbox)
# gt_class_id_list.append(gt_class_id)
# gt_mask_list.append(gt_mask)
# print("Calculating mAP over {} images".format(len(image_ids)), flush=True)
# AP, precisions, recalls, overlaps = utils.compute_ap_range(
# gt_bbox_list, gt_class_id_list, gt_mask_list,
# r['rois'], r['class_ids'], r['scores'], r['masks'],
# verbose=0)
# print("mAP: {:.3f}".format(AP))
# AP0,MeanAcc,IoMatrixAP0,MeanMIOU = compute_batch_ap(dataset, dataset.image_ids[:1000])
# print("Mean AP over {} images: {:.4f}".format(len(AP0), np.mean(AP0)))
# print("Mean Accuracy over {} images: {:.4f}".format(len(MeanAcc), np.mean(MeanAcc)))
# print("MeanMIOU over {} images: {:.4f}".format(len(MeanMIOU), np.mean(MeanMIOU)))
```

```
Loaded 1 mask(s) for image ID: 0
Loaded 1 mask(s) for image ID: 1
Loaded 1 mask(s) for image ID: 10
Loaded 1 mask(s) for image ID: 11
Loaded 1 mask(s) for image ID: 12
Loaded 1 mask(s) for image ID: 13
Loaded 1 mask(s) for image ID: 14
Loaded 1 mask(s) for image ID: 15
Loaded 1 mask(s) for image ID: 16
Loaded 1 mask(s) for image ID: 17
Loaded 1 mask(s) for image ID: 18
Loaded 1 mask(s) for image ID: 19
Loaded 1 mask(s) for image ID: 2
Loaded 1 mask(s) for image ID: 20
Loaded 1 mask(s) for image ID: 21
Loaded 1 mask(s) for image ID: 22
Loaded 1 mask(s) for image ID: 23
Loaded 1 mask(s) for image ID: 24
Loaded 1 mask(s) for image ID: 25
Loaded 1 mask(s) for image ID: 26
Loaded 1 mask(s) for image ID: 27
Loaded 1 mask(s) for image ID: 28
Loaded 1 mask(s) for image ID: 29
Loaded 1 mask(s) for image ID: 3
Loaded 1 mask(s) for image ID: 30
Loaded 1 mask(s) for image ID: 31
Loaded 1 mask(s) for image ID: 32
Loaded 1 mask(s) for image ID: 33
Loaded 1 mask(s) for image ID: 34
Loaded 1 mask(s) for image ID: 35
Loaded 1 mask(s) for image ID: 36
Loaded 1 mask(s) for image ID: 37
Loaded 1 mask(s) for image ID: 38
Loaded 1 mask(s) for image ID: 39
Loaded 1 mask(s) for image ID: 4
Loaded 1 mask(s) for image ID: 40
Loaded 1 mask(s) for image ID: 41
Loaded 1 mask(s) for image ID: 42
Loaded 1 mask(s) for image ID: 43
Loaded 1 mask(s) for image ID: 44
Loaded 1 mask(s) for image ID: 45
Loaded 1 mask(s) for image ID: 46
Loaded 1 mask(s) for image ID: 47
Loaded 1 mask(s) for image ID: 48
Loaded 1 mask(s) for image ID: 49
Loaded 1 mask(s) for image ID: 5
Loaded 1 mask(s) for image ID: 50
Loaded 1 mask(s) for image ID: 51
Loaded 1 mask(s) for image ID: 52
Loaded 1 mask(s) for image ID: 53
Loaded 1 mask(s) for image ID: 54
Loaded 1 mask(s) for image ID: 6
Loaded 1 mask(s) for image ID: 7
Loaded 1 mask(s) for image ID: 8
Loaded 1 mask(s) for image ID: 9
Loaded 1 mask(s) for image ID: 10
Loaded 1 mask(s) for image ID:
```